# Next.js

## I. Un méta-framework React et son routeur

React est une bibliothèque d'interface : il rend des composants, rien de plus. Une application complète doit choisir seule son routeur, son chargement de données, son bundler et son mode de rendu. Next.js, maintenu par Vercel, fournit ces briques déjà assemblées autour de React.

| BESOIN | SPA REACT (VITE) | NEXT.JS |
|---|---|---|
| Routage | Bibliothèque à ajouter, ou état local | Dossiers de `src/app` |
| Données | `fetch` dans `useEffect` | Composant serveur `async` |
| Rendu | Navigateur uniquement | Build (SSG), requête (SSR) ou navigateur (CSR) |
| Code serveur | Backend séparé | Dans le même projet |
| Images, polices | À la main | `next/image`, `next/font` |
| Bundler | Vite | Turbopack |

Next.js propose deux routeurs. Le **Pages Router** ( `pages/` ) est l'historique. L'**App Router** ( `app/` ) est le modèle actuel, bâti sur les React Server Components. Ce cours utilise uniquement l'App Router.

On parle de **méta-framework** : un framework construit au-dessus d'une bibliothèque de composants. Nuxt joue ce rôle pour Vue, SvelteKit pour Svelte.

Next.js - Documentation App Router · React - Creating a React App

## Quatre paradigmes du framework

**1. Le système de fichiers est le routeur.** Un dossier de `src/app` crée un segment d'URL, un fichier réservé ( `page` , `layout` , `error` , `not-found` ) lui donne un rôle. Aucune table de routes à maintenir.

**2. Serveur par défaut, client sur demande.** Chaque composant s'exécute sur le serveur. `"use client"` marque les seuls composants dont le code est transmis au navigateur.

**3. Le code décide du rendu.** Aucune donnée n'est mise en cache implicitement. `"use cache"` rend un résultat statique, `Suspense` délimite ce qui est streamé à la requête. Next.js en déduit la stratégie de chaque route.

**4. Un projet full-stack.** Le même dépôt contient l'interface et le code serveur :

- Server Functions ( `"use server"` ) : mutations appelées par un formulaire
- Route Handlers ( `route.ts` ) : endpoints HTTP, par exemple un webhook
- `proxy.ts` : code exécuté avant chaque requête (redirection, en-têtes)

Next.js - Project structure · Next.js - Mutating data

## Next.js face à Angular, SvelteKit et Nuxt

| | NEXT.JS | ANGULAR | SVELTEKIT | NUXT |
|---|---|---|---|---|
| Composants | React | Angular | Svelte | Vue |
| Routes | Dossiers `app/` | Tableau `Routes` en TS | Dossiers `routes/` | Fichiers `app/pages/` |
| Écriture serveur | Server Functions | API externe | Form actions | `server/api/` |
| Choix du rendu | Déduit du code | `app.routes.server.ts` | Options exportées | `routeRules` |
| Hydratation | Client Components seuls | Toute la page, sauf `@defer` | Toute la page | Toute la page |

Angular est un framework complet : injection de dépendances, formulaires, HTTP et tests intégrés. Il ne fournit pas de couche serveur pour écrire des données : l'application appelle une API.

SvelteKit et Nuxt suivent le même modèle que Next.js : routes par fichiers, code serveur dans le projet, déploiement sur Node.js ou sur un CDN-edge.

Angular - SSR · SvelteKit - Loading data · Nuxt - Data fetching

## Ce que Vite envoie au navigateur

Une SPA React (Vite) est rendue côté client (CSR). Le serveur envoie un HTML presque vide et un bundle JavaScript. Le contenu n'existe qu'après le téléchargement du bundle, l'exécution de React, puis la réponse de Supabase.

```
CSR (Vite)    : HTML vide → JS → React → fetch Supabase → contenu
Next.js       : contenu déjà dans le HTML → JS → interactivité
```

Un moteur de recherche, un aperçu de lien ou un téléphone lent reçoivent d'abord la version vide.

Next.js - Server and Client Components · web.dev - Rendering on the Web

## Créer le projet

`create-next-app` génère un projet TypeScript strict avec App Router, ESLint et Turbopack. Il initialise aussi un dépôt Git. Ce projet `nextjs-lab` sert de terrain d'essai pour chaque notion du cours. Le TP final repart d'un projet neuf.

```
npx create-next-app@latest nextjs-lab \
  --ts --eslint --app --src-dir --no-tailwind --no-react-compiler \
  --import-alias "@/*" --use-npm
cd nextjs-lab
npm install zod react-markdown server-only
# vitest nécessite @types/node 22 au miniumum, le template installe la version 20
npm install -D @types/node@24 vitest
npm run dev
```

Le fichier `next.config.ts` généré active `cacheComponents` et `partialPrefetching`, le modèle de rendu actuel de Next.js. Ne retirez pas ces deux options, la section III explique leur effet.

> ***TP**: créez le projet et ouvrez `http://localhost:3000`. Ouvrez `AGENTS.md` à la racine : qu'en dites vous ?*

Next.js - Installation · create-next-app

## Un dossier devient un segment d'URL

Le routeur lit l'arborescence de `src/app`. Un dossier crée un segment d'URL. Le fichier `page.tsx` rend ce segment public. Un dossier entre crochets capture une valeur.

```
src/app/
  layout.tsx                → enveloppe toutes les pages
  page.tsx                  → /
  articles/
    page.tsx                → /articles
    error.tsx               → erreur des pages /articles
    [slug]/
      page.tsx              → /articles/typescript-strict
      not-found.tsx         → slug inconnu
```

`components`, `services` et `types` restent hors de `app`. Ils ne créent aucune route.

> *Sans coder, donnez l'URL produite par `app/projects/page.tsx`, `app/projects/[slug]/page.tsx` et `app/about/page.tsx`. Que se passe-t-il pour `app/components/Button.tsx` ? Vérifiez en créant les fichiers dans votre projet.*

Next.js - Project structure

## `layout` enveloppe, `page` remplit

Le layout racine contient `<html>` et `<body>`. Il reçoit la page dans `children` et reste monté pendant la navigation.

```tsx
// src/app/layout.tsx
import Link from "next/link";
import "./globals.css";

export default function RootLayout({ children }: LayoutProps<"/">) {
  return (
    <html lang="fr">
      <body>
        <nav aria-label="Navigation principale">
          <Link href="/">Accueil</Link> <Link href="/articles">Articles</Link>
        </nav>
        {children}
      </body>
    </html>
  );
}
```

`LayoutProps<"/">` est un type global, généré par Next.js à partir de vos routes.

> 🛠️ ***TP (~5 min)*** *: ajoutez cette navigation et une page* `src/app/articles/page.tsx` *avec un* `<h1>`*. Dans le layout, modifiez les metadata avec* `metadata = { title: { template: "%s | Portfolio", default: "Portfolio" } }`*, puis* `metadata = { title: "Articles" }` *dans la page. Quel titre affiche l'onglet sur chaque page ?*

Next.js - Layouts and pages

## Typer les paramètres de route

Dans un segment `[slug]`, `params` est une `Promise`. La page la transmet sans l'attendre, un composant enfant l'attend dans `Suspense`. Le type global `PageProps`, généré par `next dev` ou `next build`, déduit les clés depuis le chemin de la route.

```tsx
// src/app/articles/[slug]/page.tsx
import { Suspense } from "react";

export default function ArticlePage({ params }: PageProps<"/articles/[slug]">) {
  return (
    <Suspense fallback={<p>Chargement...</p>}>
      <ArticleTitle params={params} />
    </Suspense>
  );
}

async function ArticleTitle({ params }: Pick<PageProps<"/articles/[slug]">, "params">) {
  const { slug } = await params;
  return <h1>Article : {slug}</h1>;
}
```

> 🛠️ ***TP (~5 min)*** *: Consultez les URL `/articles/bonjour`, puis `/articles/salut`. Dans le code, remplacez `"/articles/[slug]"` par `"/articles/[id]"` dans `PageProps`. Que répond TypeScript ? Corrigez, puis lisez `params.slug` sans `await`.*

Next.js - Dynamic routes · Next.js - TypeScript

## Une interaction devient un lien

Dans une SPA React sans routeur, un composant parent garde `selectedSlug` dans un état. Sous Next.js, chaque article possède sa propre URL.

```tsx
// src/components/ArticleCard.tsx
import Link from "next/link";

type ArticleCardProps = {
  article: { title: string; slug: string; excerpt: string };
};

export function ArticleCard({ article }: ArticleCardProps) {
  return (
    <article className="card">
      <h2>{article.title}</h2>
      <p>{article.excerpt}</p>
      <Link href={`/articles/${article.slug}`}>Lire l&apos;article</Link>
    </article>
  );
}
```

`Link` s'utilise directement dans un Server Component. Next.js précharge la page cible et navigue sans recharger le document. L'URL d'un article devient partageable, indexable et compatible avec le bouton Retour.

> 🛠️ ***TP** : créez ce composant. Dans `src/app/articles/page.tsx`, affichez trois `ArticleCard` à partir d'un tableau écrit en dur.*

Next.js - Linking and navigating

# II. Séparer le serveur et le client

## Un Server Component est une fonction asynchrone

Dans l'App Router, un composant est un **Server Component** par défaut : il s'exécute sur le serveur, jamais dans le navigateur.

- **Dans le navigateur**, React affiche le composant tout de suite, puis le redessine quand les données arrivent. D'où `useEffect`, `useState` et un état « Chargement… ».
- **Sur le serveur**, le composant s'exécute une seule fois. React attend qu'il ait fini, puis envoie la page complète. Le composant peut donc être `async` et lire ses données avec `await fetch(...)`.

```tsx
// src/app/page.tsx
type Row = { title: string };

export default async function Home() {
  "use cache"; // Build time
  const response = await fetch(
    "https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles?select=title",
    { headers: { apikey: "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV" } },
  );
  const rows: Row[] = await response.json();
  return <ul>{rows.map((row) => <li key={row.title}>{row.title}</li>)}</ul>;
}
```

Son code n'est jamais envoyé au navigateur. Seul son résultat, du HTML et une description sérialisée de l'arbre (le **RSC Payload**), voyage.

> 🛠️ ***TP (~5 min)*** *: remplacez `src/app/page.tsx` par ce composant. Ouvrez `Ctrl+U` : les titres sont-ils dans le HTML ? Lancez `npm run build`, puis `grep -rl "sb_publishable_" .next/static | wc -l`. Combien de fichiers JavaScript contiennent la clé ?*

Next.js - Fetching data

## Le RSC Payload transporte le résultat

Le RSC Payload contient :

- le résultat rendu des Server Components, sous forme de données et non de HTML ;
- des emplacements pour les Client Components, avec la référence de leur bundle JS et leurs props sérialisées.

Au premier chargement, le navigateur affiche le HTML. React lit ensuite le payload, intégré à la page, pour reconstruire l'arbre des composants. Il ne réexécute aucun Server Component : il n'a pas leur code. Seuls les Client Components sont hydratés.

Aux navigations suivantes, le navigateur ne demande que le payload de la nouvelle page, même si elle ne contient que des Server Components. React met à jour le DOM sans recharger le document. Les Client Components restés affichés, dans un layout par exemple, gardent leur état.

> 🛠️ ***TP (~5 min)*** *: ouvrez l'onglet Réseau sur* `/`*, puis cliquez sur le lien Articles. Repérez la requête dont l'URL contient* `_rsc` *et lisez sa réponse. Le document HTML a-t-il été rechargé ?*

Next.js - Server and Client Components

## `"use client"` trace une frontière

Un composant qui a besoin d'état, d'un gestionnaire d'événement ou d'une API du navigateur devient un **Client Component**. La directive se place en tête de fichier. Ce fichier et tout ce qu'il importe partent dans le bundle navigateur.

```tsx
// src/components/CopyLinkButton.tsx
"use client";

import { useState } from "react";

export function CopyLinkButton() {
  const [copied, setCopied] = useState(false);
  async function copy() {
    await navigator.clipboard.writeText(window.location.href);
    setCopied(true);
  }
  return <button type="button" onClick={copy}>{copied ? "Lien copié" : "Copier le lien"}</button>;
}
```

Un Client Component est quand même rendu en HTML côté serveur, puis **hydraté** : React y attache les gestionnaires d'événement dans le navigateur.

> 🛠️ ***TP (~5 min)*** *: ajoutez ce bouton dans* `src/app/articles/[slug]/page.tsx` *et testez-le. Retirez ensuite la ligne* `"use client"` *et lancez* `npm run build` *.*

Next.js - use client

## Placer la frontière le plus bas possible

| BESOIN DU COMPOSANT | TYPE |
| --- | --- |
| Lire Supabase, une clé, un fichier serveur | Server |
| Afficher des props, du Markdown, une image | Server |
| `useState`, `useEffect`, `onClick`, `onChange` | Client |
| `window`, `localStorage`, `navigator.clipboard` | Client |

Plus la frontière est basse, plus le bundle est léger. Une page entière marquée `"use client"` renvoie tout son code au navigateur, y compris ce qui n'a aucune interactivité.

Next.js - When to use Server and Client Components

## Ce qui traverse la frontière doit être sérialisable

Un Server Component passe ses props au Client Component à travers le réseau. Seules les valeurs sérialisables passent : chaînes, nombres, tableaux, objets simples, `Date`, `Map`, `Promise`. Une fonction ne passe pas.

`structuredClone` se comporte de la même façon et permet de tester hors NextJS :

```
type Props = Record<string, unknown>;

function check(label: string, props: Props) {
  try {
    structuredClone(props);
    console.log("✓", label);
  } catch (error) {
    console.log("x", label, error instanceof Error ? error.name : error);
  }
}

check("article", { article: { id: 1, title: "TS", cover_url: null } });
check("date", { publishedAt: new Date("2026-09-24") });
check("onSelect", { onSelect: (slug: string) => console.log(slug) });
```

Next.js refuse la même prop au build : `Event handlers cannot be passed to Client Component props`.

> *À votre avis, si on ajoute à `CopyLinkButton` une prop `onCopied: () => void`, que se passe-t-il ?*

Next.js - Passing data from Server to Client Components

## Garder les variables d'environnement sur le serveur

Next.js lit `.env.local` dans `process.env`. Sans préfixe, une variable n'existe que côté serveur. Avec `NEXT_PUBLIC_`, sa valeur est recopiée en clair dans le bundle navigateur au moment du build.

```
# .env.local
SUPABASE_URL=https://<PROJECT_REF>.supabase.co
SUPABASE_PUBLISHABLE_KEY=sb_publishable_<KEY>
```

L'application lit Supabase depuis des Server Components. Aucune variable n'a besoin du préfixe. Même la clé publishable reste sur le serveur : le navigateur ne contacte jamais Supabase.

Le `.gitignore` généré ignore `.env*`, donc aussi `.env.example`. Ajoutez la ligne `!.env.example` pour versionner ce modèle.

Next.js - Environment variables

## Interdire un module au navigateur

Un fichier qui lit un secret ne doit jamais être importé par un Client Component. Le paquet `server-only` transforme cette erreur en échec de build.

```ts
// src/services/supabase.ts
import "server-only";
import { z } from "zod";

const envSchema = z.object({
  SUPABASE_URL: z.url(),
  SUPABASE_PUBLISHABLE_KEY: z.string().startsWith("sb_publishable_"),
});

const env = envSchema.parse(process.env);

export const apiHeaders = { apikey: env.SUPABASE_PUBLISHABLE_KEY };

export function restUrl(table: string): URL {
  return new URL(`/rest/v1/${table}`, env.SUPABASE_URL);
}
```

Importé depuis un fichier `"use client"`, ce module arrête le build : `'server-only' cannot be imported from a Client Component module`.

> 🛠️ ***TP** : créez ce service et le fichier `.env.local` avec l'URL et la clé de votre projet. Dans `src/app/page.tsx`, remplacez l'URL et la clé écrites en dur par l'usage de `restUrl("articles")` et `apiHeaders`. Comment récupérer les pages plutot que les articles ?*

Next.js - Preventing environment poisoning · Supabase - Clés API

# III. Choisir où et quand rendre

## Trois moments de rendu

| MOMENT | STRATÉGIE | EXEMPLE DANS L'APPLICATION |
|---|---|---|
| Build | SSG, page statique | Accueil, article déjà publié |
| Requête | SSR, rendu à la demande | Résultats d'une recherche `?q=` |
| Navigateur | CSR | Bouton « Copier le lien » |

Next.js combine les trois dans une même page. La fin de `npm run build` indique le résultat pour chaque route :

```
Route (app)                              Revalidate  Expire
┌ ○ /                                       1m      1h
├ ◐ /articles
└ ○ /articles/typescript-strict            1h      1d

○  (Static)             prerendered as static content
◐  (Partial Prerender)  prerendered as static HTML with dynamic server-streamed content
```

`○` désigne une page entièrement produite au build. `◐` désigne une page statique hydratée et complétée à chaque requête.

> *Lancez `npm run build` dans votre projet. Relevez le symbole de chaque route et expliquez-le en une phrase.*

Next.js - Caching · Patterns.dev - Rendering patterns

## Dynamique par défaut, statique par choix

Avec `cacheComponents`, aucune donnée n'est mise en cache implicitement. Au build, Next.js prérend chaque route. Il s'arrête sur une donnée lue à la requête si rien n'indique comment la traiter.

```tsx
// src/app/articles/page.tsx (version refusée au build)
export default async function ArticlesPage({ searchParams }: PageProps<"/articles">) {
  const { q } = await searchParams;
  return <h1>Recherche : {q}</h1>;
}
```

```
Error: Route "/articles": Next.js encountered uncached or runtime data during prerendering.

Ways to fix this:
  - [stream] Provide a placeholder with `<Suspense fallback={...}>` around the data access
  - [cache] For uncached data (`fetch`, database calls): cache the access with `"use cache"`
  - [block] Set `export const instant = false` to allow a blocking route
```

`searchParams`, `params`, `cookies()`, `headers()` et un `fetch` non mis en cache sont concernés. Les deux premières solutions sont les plus courantes : streamer ou mettre en cache.

> 🛠️ *remplacez `src/app/articles/page.tsx` par cette page et lancez `npm run build`. Retrouvez la route fautive dans le message. Gardez le fichier, on va corriger en suite.*

Next.js - blocking-prerender-dynamic

## Streamer avec `Suspense`

`<Suspense>` délimite une zone qui peut attendre. Le reste de la page part dans le contenu statique avec le `fallback`. Le contenu de la zone suit dans la même réponse HTTP.

```tsx
// src/app/articles/page.tsx
import { Suspense } from "react";

export default function ArticlesPage({ searchParams }: PageProps<"/articles">) {
  return (
    <main>
      <h1>Articles</h1>
      <Suspense fallback={<p role="status">Chargement...</p>}>
        <SearchSummary searchParams={searchParams} />
      </Suspense>
    </main>
  );
}

async function SearchSummary({ searchParams }: Pick<PageProps<"/articles">, "searchParams">) {
  const { q } = await searchParams;
  return <p>Recherche : {typeof q === "string" ? q : "aucune"}</p>;
}
```

> 🛠️ ***TP (~5 min)*** *: corrigez votre page avec ce modèle, relancez le build et notez le nouveau symbole de* `/articles`.

React - Suspense · Next.js - Streaming

## Décrire les données avec Zod

Les services valident chaque réponse de Supabase. Le schéma décrit une ligne de la table `articles`, les autres types en dérivent.

```ts
// src/types/article.ts
import { z } from "zod";

export const articleSchema = z.object({
  id: z.number().int(),
  title: z.string(),
  slug: z.string(),
  excerpt: z.string(),
  content: z.string().nullable(),
  cover_url: z.string().nullable(),
  cover_alt: z.string().nullable(),
  published_at: z.iso.datetime({ offset: true }),
});

export const articlePreviewSchema = articleSchema.pick({
  id: true, title: true, slug: true, excerpt: true, cover_url: true, cover_alt: true,
});

export const articleListSchema = z.array(articleSchema);
export const articlePreviewListSchema = z.array(articlePreviewSchema);
export const articleSlugListSchema = z.array(articleSchema.pick({ slug: true }));

export type Article = z.infer<typeof articleSchema>;
export type ArticlePreview = z.infer<typeof articlePreviewSchema>;
export type ArticlePage = { articles: ArticlePreview[]; total: number };
```

> 🛠️ ***TP (~3 min)*** *: créez* `src/types/article.ts`.

## Mettre en cache avec `"use cache"`

La directive `"use cache"` en tête d'une fonction asynchrone mémorise son résultat. `cacheLife` fixe sa durée de vie, `cacheTag` lui attache une étiquette pour l'invalider à la demande.

```ts
// src/services/articles.ts
import { cacheLife, cacheTag } from "next/cache";
import { articleListSchema, articleSlugListSchema, type Article } from "@/types/article";
import { apiHeaders, restUrl } from "./supabase";

export async function fetchArticleBySlug(slug: string): Promise<Article | null> {
  "use cache";
  cacheLife("hours");
  cacheTag("articles", `article-${slug}`);

  const url = restUrl("articles");
  url.searchParams.set("select", "id,title,slug,excerpt,content,cover_url,cover_alt,published_at");
  url.searchParams.set("slug", `eq.${slug}`);
  const response = await fetch(url, { headers: apiHeaders });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const result = articleListSchema.safeParse(await response.json());
  if (!result.success) throw new Error("Invalid article response");
  // A missing slug and a draft hidden by RLS both return [].
  return result.data[0] ?? null;
}

export async function fetchArticleSlugs(): Promise<string[]> {
  "use cache";
  cacheLife("hours");
  cacheTag("articles");

  const url = restUrl("articles");
  url.searchParams.set("select", "slug");
  const response = await fetch(url, { headers: apiHeaders });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const result = articleSlugListSchema.safeParse(await response.json());
  if (!result.success) throw new Error("Invalid slugs response");
  return result.data.map((row) => row.slug);
}
```

> 🛠️ ***TP (~3 min)*** *: créez* `src/services/articles.ts` *.*

| PROFIL | RAFRAÎCHI APRÈS | USAGE |
|---|---|---|
| `minutes` | 1 minute | Liste, recherche |
| `hours` | 1 heure | Article publié |
| `days` | 1 jour | Page « À propos » |

Les arguments font partie de la **clé de cache**. `fetchArticleBySlug("a")` et `fetchArticleBySlug("b")` occupent deux entrées distinctes. Un argument doit donc être sérialisable (chaîne, nombre, objet simple). Une fonction ou un objet vivant ne l'est pas.

Next.js – use cache · Next.js – cacheLife

## Prérendre les articles connus

`generateStaticParams` liste les valeurs de `[slug]` à produire au build. Chaque article publié devient une page statique. Un slug inconnu est rendu à la première visite, puis mis en cache.

```tsx
// src/app/articles/[slug]/page.tsx
import { notFound } from "next/navigation";
import { Suspense } from "react";
import { fetchArticleBySlug, fetchArticleSlugs } from "@/services/articles";

export async function generateStaticParams() {
  const slugs = await fetchArticleSlugs();
  return slugs.map((slug) => ({ slug }));
}

export default function ArticlePage({ params }: PageProps<"/articles/[slug]">) {
  return (
    <Suspense fallback={<p role="status">Chargement...</p>}>
      <ArticleContent params={params} />
    </Suspense>
  );
}

async function ArticleContent({ params }: Pick<PageProps<"/articles/[slug]">, "params">) {
  const { slug } = await params;
  const article = await fetchArticleBySlug(slug);
  if (article === null) notFound();
  return <h1>{article.title}</h1>;
}
```

`notFound()` affiche `not-found.tsx`. Ensuite, TypeScript sait que `article` n'est plus `null`. Avec `cacheComponents`, `generateStaticParams` doit renvoyer au moins une valeur.

> 🛠️ ***TP (~5 min)*** *: lancez `npm run build`. Dans la sortie, repérez les lignes sous `/articles/[slug]`. Comparez au nombre d'articles publiés dans Supabase.*

Next.js - generateStaticParams · Next.js - notFound

## Garder `params` sous `Suspense`

Au build, Next.js prépare une partie statique de la route : layout, titre, fallback. Toutes les URL `/articles/*` partagent cette partie statique, ce qui rend la navigation instantanée.

Le slug ne doit donc être lu que dans un composant enveloppé par `Suspense`, comme `ArticleContent` à la diapo précédente. Si la page fait elle-même `await params`, toute la page dépend de l'URL : plus de partie statique commune.

```tsx
// src/app/articles/[slug]/page.tsx (version signalée en développement)
export default async function ArticlePage({ params }: PageProps<"/articles/[slug]">) {
  const { slug } = await params;
  const article = await fetchArticleBySlug(slug);
  if (article === null) notFound();
  return <h1>{article.title}</h1>;
}
```

Le build ne signale rien. Seul `next dev` affiche une erreur au chargement :

```
Error: Route "/articles/[slug]": Next.js encountered URL data during prerendering or a navigation.

`params` or `searchParams` accessed outside of `<Suspense>` may prevent the navigation
from being instant, leading to a slower user experience.
```

Ouvrez chaque route en développement avant de livrer.

> 🛠️ ***TP (~3 min)*** *: déplacez `await params` dans `ArticlePage`, rechargez `/articles/<un-slug>` en développement et lisez la console.*

Next.js - Partial Prefetching

## Conséquence : `200` au lieu de `404`

Le statut HTTP part dans les en-têtes, avant le contenu. Avec `Suspense`, Next.js envoie la partie statique tout de suite, donc avec un `200`.

`notFound()` s'exécute ensuite dans `ArticleContent`. Le `200` est déjà parti : impossible de le changer en `404`.

Next.js compense en ajoutant `<meta name="robots" content="noindex">`. La page « introuvable » s'affiche et les moteurs de recherche ne l'indexent pas.

C'est le prix à payer : une navigation instantanée contre un vrai statut `404`.

> 🛠️ ***TP (~2 min)*** *: lancez* `curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/articles/inconnu` *, puis* `curl -s http://localhost:3000/articles/inconnu | grep -c noindex` *.*

Next.js - Streaming et codes de statut

## Parallèle Angular : un mode de rendu par route

Angular déclare la stratégie route par route, dans un fichier serveur séparé. Next.js la déduit du code de chaque composant ( `"use cache"` , `Suspense` , API de requête).

```ts
// src/app/app.routes.server.ts
import { inject } from "@angular/core";
import { RenderMode, ServerRoute } from "@angular/ssr";
import { ArticleService } from "./article.service";

export const serverRoutes: ServerRoute[] = [
  { path: "", renderMode: RenderMode.Prerender },
  { path: "articles", renderMode: RenderMode.Server },
  {
    path: "articles/:slug",
    renderMode: RenderMode.Prerender,
    async getPrerenderParams() {
      const service = inject(ArticleService);
      const slugs = await service.getSlugs();
      return slugs.map((slug) => ({ slug }));
    },
  },
];
```

Autre différence de modèle : Angular hydrate les composants rendus côté serveur, sauf les blocs `@defer` dotés d'un déclencheur `hydrate` . Next.js n'envoie au navigateur que le code des Client Components.

Angular - SSR et rendu hybride · Angular - Incremental hydration

# IV. Construire la liste d'articles

### Mettre le service en cache

Le service vérifie le statut HTTP, valide le JSON par Zod et lit le total dans `Content-Range`. `"use cache"` et `cacheLife` en font une donnée statique.

Ajoutez `src/services/pagination.ts` (taille de page, lecture du total, nombre de pages), puis `buildArticlesUrl` et `fetchArticles` dans `src/services/articles.ts`.

```
// src/services/pagination.ts
export const PAGE_SIZE = 6;

// Content-Range looks like "0-5/13": rows 0 to 5, 13 rows in total.
export function readTotal(contentRange: string | null): number {
  const total = Number(contentRange?.split("/")[1]);
  if (!Number.isInteger(total) || total < 0) throw new Error("Missing total count");
  return total;
}

export function countPages(total: number): number {
  return Math.max(1, Math.ceil(total / PAGE_SIZE));
}
```

```
// src/services/articles.ts (ajouts ; imports : articlePreviewListSchema, ArticlePage, PAGE_SIZE, readTotal)
export function buildArticlesUrl(query: string, page: number): URL {
  const url = restUrl("articles");
  url.searchParams.set("select", "id,title,slug,excerpt,cover_url,cover_alt");
  url.searchParams.set("order", "created_at.desc,id.desc");
  url.searchParams.set("limit", String(PAGE_SIZE));
  url.searchParams.set("offset", String((page - 1) * PAGE_SIZE));
  const term = query.trim();
  if (term) url.searchParams.set("title", `ilike.*${term}*`);
  return url;
}

export async function fetchArticles(query: string, page: number): Promise<ArticlePage> {
  "use cache";
  cacheLife("minutes");
  cacheTag("articles");

  const headers = { ...apiHeaders, Prefer: "count=exact" };
  const response = await fetch(buildArticlesUrl(query, page), { headers });
  // 416: the requested page is past the last row. The total is still sent.
  if (response.status === 416) {
    return { articles: [], total: readTotal(response.headers.get("Content-Range")) };
  }
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const result = articlePreviewListSchema.safeParse(await response.json());
  if (!result.success) throw new Error("Invalid articles response");
  return { articles: result.data, total: readTotal(response.headers.get("Content-Range")) };
}
```

Un numéro de page au-delà de la dernière, saisi dans l'URL (`?page=99` pour 13 articles), renvoie `416`. Le service ne lève pas d'erreur : il renvoie une liste vide et le nombre total d'articles, lu dans `Content-Range`. La page liste utilise ce total pour répondre `404` (voir « Assembler la page liste »).

Exemple:

```
const url = "https://nxfqfvxpdsqsskobqent.supabase.co/rest/v1/articles?select=title&offset=60";
const response = await fetch(url, {
  headers: {
    apikey: "sb_publishable_aaodWttdxy_9cC5CSr6f8Q_iOPRcbGV",
    Prefer: "count=exact",
  },
});
console.log(response.status, response.headers.get("Content-Range"));
```

Next.js – Fetching data · PostgREST – Exact count

## L'URL remplace l'état

Dans une SPA, `query` et `page` vivent dans `useState`. Ici, ils vivent dans `?q=...&page=...`. `src/services/list-params.ts` traduit dans les deux sens : URL vers paramètres pour lire la page, paramètres vers URL pour les liens de pagination. Une URL est une saisie utilisateur : elle se valide comme une réponse d'API.

```ts
import { z } from "zod";

const listParamsSchema = z.object({
  q: z.string().trim().default(""),
  page: z.coerce.number().int().min(1).default(1),
});

type ListParams = { query: string; page: number };

function parseListParams(raw: Record<string, string | string[] | undefined>): ListParams {
  const result = listParamsSchema.safeParse(raw);
  return result.success ? { query: result.data.q, page: result.data.page } : { query: "", page: 1 };
}

function articlesHref({ query, page }: ListParams): string {
  const params = new URLSearchParams();
  if (query) params.set("q", query);
  if (page > 1) params.set("page", String(page));
  const search = params.toString();
  return search ? `/articles?${search}` : "/articles";
}

console.log(parseListParams({ q: " zod ", page: "2" })); // { query: "zod", page: 2 }
console.log(parseListParams({ q: ["a", "b"], page: "abc" })); // { query: "", page: 1 }
console.log(articlesHref({ query: "React & Zod", page: 2 })); // /articles?q=React+%26+Zod&page=2
```

`safeParse` ne lève jamais d'erreur. Si un champ est invalide, la fonction renvoie la première page sans recherche. Avec `parse`, une URL bricolée comme `?page=abc` lèverait une `ZodError` et la page tomberait sur `error.tsx`. `.default(valeur)` ne remplit qu'un champ absent.

> 🛠️ ***TP** : testez `page: "-3"`, `"1.5"` et `"1e3"`, puis ajoutez `max(500)`.*

- `"-3"` : `1`, `min(1)` échoue et `safeParse` renvoie la valeur de repli.
- `"1.5"` : `1`, `int()` échoue.
- `"1e3"` : `1000`, passe. `Number("1e3")` vaut `1000`, un entier positif. Avec `max(500)`, il donne `1`.

```ts
page: z.coerce.number().int().min(1).max(500).default(1),
```

-->

Zod - Coercion · Zod - Default · MDN - URLSearchParams

## Rechercher et paginer sans état

`next/form` étend `<form>` : la soumission met à jour l'URL par une navigation client, sans recharger le document. Le formulaire fonctionne aussi sans JavaScript.

```tsx
// src/components/SearchForm.tsx
import Form from "next/form";

export function SearchForm({ query }: { query: string }) {
  return (
    <Form action="/articles" className="search" role="search">
      <label htmlFor="article-search">Rechercher un article</label>
      <input id="article-search" type="search" name="q" defaultValue={query} />
      <button type="submit">Rechercher</button>
    </Form>
  );
}
```

```tsx
// src/components/Pagination.tsx (extrait)
const pageCount = countPages(total);
// ...
{page > 1 && <Link href={articlesHref({ query, page: page - 1 })}>Page précédente</Link>}
<span>Page {page} sur {pageCount}</span>
{page < pageCount && <Link href={articlesHref({ query, page: page + 1 })}>Page suivante</Link>}
```

Le formulaire `GET` écrit `?q=...` sans le paramètre `page` : une nouvelle recherche revient à la page 1 sans aucun code. Aucun de ces composants n'a besoin de `"use client"`.

Next.js - Form · Next.js - Link

## Assembler la page liste

Le titre reste dans la partie statique. Les résultats attendent l'URL dans `Suspense`. Une page au-delà de la dernière donne `404` : `countPages` (`src/services/pagination.ts`) calcule le nombre de pages.

```tsx
// src/app/articles/page.tsx (extrait)
export default function ArticlesPage({ searchParams }: PageProps<"/articles">) {
  return (
    <main>
      <h1>Articles</h1>
      <Suspense fallback={<p role="status">Chargement des articles...</p>}>
        <ArticleResults searchParams={searchParams} />
      </Suspense>
    </main>
  );
}

async function ArticleResults({ searchParams }: Pick<PageProps<"/articles">, "searchParams">) {
  const { query, page } = parseListParams(await searchParams);
  const { articles, total } = await fetchArticles(query, page);
  if (page > countPages(total)) notFound();
  return (
    <>
      <SearchForm key={query} query={query} />
      {articles.length === 0
        ? <p>Aucun article ne correspond à la recherche.</p>
        : <><ArticleList articles={articles} /><Pagination query={query} page={page} total={total} /></>}
    </>
  );
}
```

`key={query}` recrée le champ quand la recherche change : `defaultValue` suit l'URL. Next.js - page.js

## Optimiser la couverture avec `next/image`

`next/image` redimensionne l'image et la convertit en WebP si le navigateur l'accepte. `width` et `height` réservent sa place et évitent un décalage de mise en page (CLS).

```tsx
// src/components/ArticleCover.tsx
import Image from "next/image";

export function ArticleCover({ url, alt }: { url: string | null; alt: string | null }) {
  if (url === null) return null;
  return (
    <Image className="cover" src={url} alt={alt ?? ""} width={1200} height={675}
      sizes="(max-width: 68rem) 100vw, 68rem" />
  );
}
```

```ts
// next.config.ts: a remote image must be allowed explicitly
images: {
  remotePatterns: [
    { protocol: "https", hostname: "*.supabase.co", pathname: "/storage/v1/object/public/**" },
  ],
},
```

> 🛠️ ***TP (~5 min)*** *: retirez `remotePatterns`, rechargez une page avec couverture et lisez le message. Remettez l'option, puis ouvrez l'onglet Réseau : quel format et quelle largeur reçoit le navigateur ?*

Next.js - Images · Next.js - Image

## Remplacer les états d'erreur par des fichiers

Dans une SPA, un hook expose `loading`, `error` et `success`. Next.js répartit ces états entre trois mécanismes.

- `Suspense` affiche le chargement.
- `not-found.tsx` affiche l'absence de contenu, après `notFound()`.
- `error.tsx` capture une exception levée pendant le rendu du segment.

```tsx
// src/app/articles/error.tsx
"use client";

export default function ArticlesError({ error, retry }: { error: Error & { digest?: string }; retry: () => void }) {
  return (
    <main>
      <h1>Articles indisponibles</h1>
      <p role="alert">Le chargement a échoué : {error.message}</p>
      <button type="button" onClick={() => retry()}>Réessayer</button>
    </main>
  );
}
```

`error.tsx` est un Client Component : `retry()` relance le rendu depuis le navigateur. En production, `error.message` est remplacé par un message générique pour ne pas exposer de détail serveur.

> 🛠️ ***TP (~5 min)*** *: en développement, remplacez la clé dans `.env.local` par `sb_publishable_wrong` et ouvrez `/articles`. Quel fichier s'affiche ? Remettez la clé, puis cliquez sur « Réessayer ».*

Next.js - Error handling · Next.js - error.js